

Comparison of NoSQL Datastores for Large Scale Data Stream Log Analytics

Khalid Mahmood

Department of Information Technology
Uppsala University
Uppsala, Sweden
Khalid.Mahmood@it.uu.se

Kjell Orsborn

Department of Information Technology
Uppsala University
Uppsala, Sweden
Kjell.Orsborn@it.uu.se

Tore Risch

Department of Information Technology
Uppsala University
Uppsala, Sweden
Tore.Risch@it.uu.se

Abstract—With the advent of cyber-physical systems, industrial internet of things (IIoT) and industrial analytics numerous application scenarios have emerged where business and mission-critical decisions depend upon large scale analysis of data in form of sensor streams. However, large volumes of sensor stream data generated at high frequency pose substantial challenges for existing scalable data analysis techniques requiring the use of high-performance distributed datastores. This work covers in-depth performance comparison of three principal categories of distributed state-of-the-art NoSQL datastores by evaluating their applicability and efficiency for large-scale analysis of sensor logs from real-world hydraulic power systems. One central datastore is selected from each of the three principal categories of NoSQL datastores: MongoDB from the *document store*, Cassandra from the *column store* and Redis from the *distributed main memory key-value store* to be included in the performance evaluation. Understanding the differences and behavior of this type of systems are crucial for optimizing application performance. Key insights from this work can serve as a basis for an improved understanding of the applicability of NoSQL datastores in systems for large scale data stream analysis. This will be important for supporting data analytics in IIoT applications as found in monitoring and control of Power plants, Smart Cities, Transportation systems, Environmental and Health monitoring, etc.

Index Terms—NoSQL Datastores, IIoT, Smart Computing

I. INTRODUCTION

The recent scientific and technological advances within the areas of cyber-physical systems, industrial internet of things (IIoT) and industrial analytics have raised the need to handle large quantities of data generated at high rates from IIoT sensor systems in power plants, transportation systems and other types of industrial equipment. Many data analytics tasks in these types of applications include large scale analysis of data from sensor streams in form of sensor stream record logs. Various industrial scenarios have been identified in our involvement in the Smart Vortex project [1] (a collaborative project involving European universities and manufacturing companies as e.g. Volvo CE, Bosch Rexroth and Sandvik). Data streams from fleets of vehicles, machining equipment and hydraulic power systems are examples of industrial data analytics use cases that all involve large scale analysis of sensor data. A variety of industrial processes can be monitored and controlled resulting in cost reductions by for instance

This work was supported by the Swedish research programme eSENCE.

detecting machine failures at an early stage to support planned maintenance instead of catastrophic failures. If a hydraulic power system is supporting an industrial mining process the difference between planned maintenance and catastrophic failures can be counted in millions of Euros.

A major challenge of smart computing is efficiently performing scalable analysis of massive volumes of sensor stream logs [2]. As the data rate of incoming streams of sensors can be extremely high, the volume of stream logs quickly become very large as well. An example of such a scenario is illustrated in Fig. 1 where data streams are captured from pressure sensors from a small fleet of hydraulic power systems for the last hour of production time. The key property that this type of applications exhibit is that there could be millions of records originating from different types of sensors (e.g. temperature, pressure, power, etc.) being continuously logged from hundreds of machines that need to be analyzed.

Typically, relational databases are used for large-scale analysis of persisted streams [3] where analytical tasks are expressed in the SQL query language. Consider the following simplified SQL query, which identifies the abnormal behavior of different machine quantities in a smart factory by comparing if the actual measurements from sensors have exceeded the expected threshold. Exceeding the maximum threshold might indicate that the machine will require immediate maintenance in order to avoid unplanned disturbance of production.

```
SELECT DISTINCT machines FROM measurements
WHERE timestamp > (time.now() - one_hour)
AND measured_pressure > @pressure_threshold
```

Two important and major types of basic tasks involved in analytical SQL queries are categorized as:

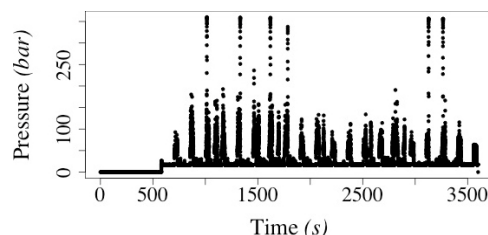


Fig. 1: Pressure measurements of sensors for 1 hour

Key lookup query: this type of query task is used to find a specific sensor log from a large data sets.

Range query: this type of query task retrieves all records between some upper and lower bounds from the dataset.

Despite being primitive, both types of queries are nontrivial and serve as building blocks of many more advanced and complex queries, therefore, considered as fundamental queries for analysis of stream logs [3], [4].

A potential problem for executing these fundamental queries over large volumes of data logs using a conventional relational database management system (RDBMS) is that loading massive logs produced at high rates are not fast enough due to the strong consistency model and a high cost of indexing [5]. An additional challenge arises since the RDBMS is typically deployed centrally which is prone to a single point of failure that could be fatal for mission-critical applications. On the other hand, the sheer volume of data makes a single server storage incapable of scaling such analysis of fundamental queries. In order to alleviate the limitation of conventional RDBMS, a modern distributed NoSQL datastore can be utilized to achieve higher performance and scalability. These datastores can be deployed over many machines thus have no single point of failure [6]. There are primarily three major categories of NoSQL datastores [6], [7] (a) *Column Store*: partitions data vertically and horizontally across nodes. (b) *Key-Value Store*: uses a data model of single key-value indexes for all the data. (3) *Document Store*: supports more complex data representation than the key-value stores. These type of datastores are also available in major cloud services provided in e.g. Amazon Web Services and Google Cloud Platform [8]. Therefore, to what degree state-of-the-art NoSQL datastore can support high-performance processing of fundamental queries, could essentially provide significant insights for potential capability for large scalable data analysis.

There are several benchmarks that compare the performance of NoSQL datastores, however, they are either limited to a single NoSQL to RDBMS comparison [9] or cover scalability experimentation for interactive web applications [10], [11]. These papers do not consider the option for exploring and finding the trade-off between choosing different kinds of NoSQL datastores as well as benchmarking these systems on real IoT workloads from industrial sensor streams.

In this paper, we compare one of the central system from each of the three categories of NoSQL datastores such as MongoDB (a document store), Cassandra (a column store) and Redis (a distributed key-value store). These three systems have been chosen based on the following factors, (a) open source and possible to perform cross-vendor comparison (compared to e.g. Amazon DynamoDB and Google Big Table that are limited to operate only on their proprietary platform) (b) widely perform better than competitors or used previously in big data environments [9], [10], [11] (c) most popular NoSQL datastore from each category [7].

In summary, the main contribution of this research is a performance comparison of the suitability and applicability of major categories of distributed NoSQL systems for large

scale analysis of sensor logs. The analysis currently utilize over 100 millions of sensor logs from real IIoT applications that serve as a basis to better understand the trade-off between choosing different kinds of systems for fundamental queries that typically appear in large scale IoT applications.

II. PERFORMANCE EVALUATIONS

The application scenario involves analyzing data logs of in-production industrial sensors from hydraulic power systems. In this scenario, a factory operates industrial equipment that is powered by several hydraulic power systems where each equipment is installed with sensors that measure physical quantities like pressure, power, temperature, etc. For instance, logs of *measurements* are generated for sensor systems in each separate equipment and system, where each log record has a *machine identifier m*, *sensor identifier s*, and a *measured value mv*. Each measured value *mv* on machine *m* is associated with a valid time interval [*bt*, *et*] indicating the begin time and end time for *mv*. The table measures (*m*, *s*, *bt*, *et*, *mv*) will contain a large volume of log from many sensors on different machines corresponds to more than 100 million log measurements.

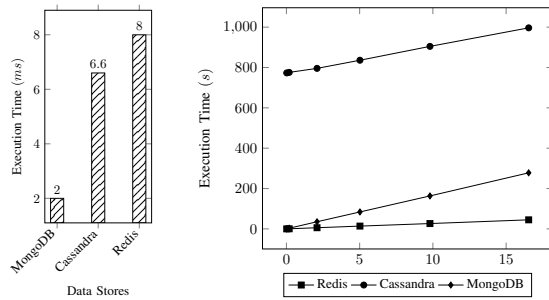
To demonstrate the credibility of the performance trade-offs for the NoSQL systems, the experimental tasks should be designed in a simplistic approach. This is one of the main criteria for domain specific benchmarks according to Gray [12]. The following two fundamental queries to analytics over numerical logs, providing basic building blocks for analytics over stream logs, are used in the evaluation [3].

```
Key Lookup Query (Q1):
SELECT * FROM measures
WHERE m =? AND s =? AND bt =?
Range Query (Q2):
SELECT * FROM measures WHERE mv > ?
```

Both of these queries will be expressed in the query languages of the corresponding datastore and executed over the data set. We have used a cluster of 64-CPU machines of an aggregated 384GB of physical memory with Scientific Linux operating system. The latest version of MongoDB, Cassandra and Redis are used for the performance investigation.

1) *Performance of the Key Lookup Query, Q1*: This task involves finding a particular log for a given machine *m*, sensor *s*, and begin time, *bt*. To provide scalable performance of *Q1*, we used the composite primary key on (*m*, *s*, *bt*) of corresponding datastores. Fig. 2a demonstrates that all system provide comparable performance as they all run under 10 milliseconds. Both Redis and Cassandra uses partitioning technique similar to *consistent hashing* while MongoDB uses *range-based partitioning* in order to distribute the data-set. Both partitioning techniques work equally well for *Q1* since a single hash function called can be used to find the record.

2) *Performance of the Range Query, Q2*: This query involves finding unexpected sensors readings by observing measured values *mv* that deviate from an expected value. Here, the sensor readings with measured values *mv* not close to the expected value are retrieved. To improve the performance of *Q2*, we use a secondary ordered index on *mv* which in the case



(a) Key Lookup Query

(b) Range Query

Fig. 2: Query Execution

of MongoDB a *b-tree index* while for Cassandra an *inverted index*. Whereas Redis, being a main-memory datastore, we used the in-memory *sorted set*. Since the efficiency of a secondary index is dependent on the selectivity, the query was executed with different query selectivities by determining the appropriate ranges of *mv* which is depicted in Fig. 2b. Clearly the in-memory datastore Redis outperforms both the disk-based NoSQL datastore Cassandra and MongoDB. Whereas, MongoDB performs better compared to Cassandra as its *b-tree index* often resides in memory causing less disk access.

III. DISCUSSION AND CONCLUSION

A number of interesting conclusions can be drawn from the experimental results where various differences in performance are revealing trade-offs in the design principles of each individual NoSQL datastore.

In the first type of task, all systems perform well for finding a single record matching the key which is executed by the *key lookup* query. This task is leveraged by utilizing a primary key index supplied by each system. In the case of Cassandra and Redis, a primary index was supported by the use of a *consistent hashing* type techniques, while for MongoDB a *range-based hashing* technique was used. This essentially demonstrates that both of these hashing techniques are equally capable of finding a specific record from a large data set. Internally, a hash value, based on the key attribute determines which node is responsible for retrieving the specific record. Then the request is dispatched to the specific server node for further processing of the task and retrieving the result for the client. These steps are included in all three systems and, therefore, they exhibit equivalent performance for the key lookup task.

The second type of task involving a range query reveals more interesting design trade-offs for the three types of systems. It is clearly noticeable that the *range-based hashing* technique and the *in-memory B-tree* indexing type used in MongoDB provide performance advantages compared to Cassandra which uses a disk-based *inverted index* for executing range queries. However, the *range-based partitioning* in MongoDB is also more prone to skewed load balancing when the distribution of the partition key is biased.

Apart from data partitioning, in-memory datastores, such as Redis, have demonstrated significant performance advantages

over the disk-based counterpart such as MongoDB and Cassandra. Though Redis uses a *consistent hashing* type technique, to compare with the *range-based hashing* of MongoDB, the in-memory data structures of Redis plays a significant role in enabling such performance boost. However, this performance advantage comes at a cost of sacrificing the persistent guarantees provided by disk-based approaches which could be prone to data loss that is critical in many applications.

We acknowledge these performance trade-offs while dealing with distributed state-of-art NoSQL systems. In fact, building large scale smart applications involve finding the balance among such trade-offs. Interestingly, however, an application can leverage by using multiple NoSQL systems in a single deployment environment. Moreover, major cloud service providers such as AWS and Google Cloud Platform offer multiple NoSQL systems from all three of these categories [8]. This, of course, requires advanced insights on how to apply these technologies. Hence, the design underpinnings of specific datastore architectures can significantly influence query-execution performance. Understanding the differences and behaviour of these type of systems is crucial for balancing such trade-offs and optimizing application performance.

In our opinion there is a lot more to learn in evaluating these types of systems. Future research will focus on in-depth performance analysis in order to reveal advantages and disadvantages originating in supported features and architectural differences in NoSQL datastores. An interesting next step in this research will be to employ billion sensor readings from streams in real-world industrial applications in order to understand the scalability issues in massive-scale data analysis, a central problem to support high-performance and energy-efficiency in smart IoT applications of the future.

REFERENCES

- [1] "Smart Vortex European Union Project," <http://www.smartvortex.eu/>.
- [2] J. Gubbi, R. Buyya, S. Marusic, and M. Palaniswami, "Internet of things (iot): A vision, architectural elements, and future directions," *Future generation computer systems*, vol. 29, no. 7, pp. 1645–1660, 2013.
- [3] M. Zhu, K. Mahmood, and T. Risch, "Scalable queries over log database collections," in *British International Conference on Databases*. Springer, 2015, pp. 173–185.
- [4] K. Mahmood, T. Risch, and M. Zhu, "Utilizing a nosql data store for scalable log analysis," in *Proceedings of the 19th International Database Engineering & Applications Symposium*. ACM, 2015, pp. 49–55.
- [5] M. Stonebraker, "Sql databases v. nosql databases," *Commun. ACM*, vol. 53, no. 4, pp. 10–11, Apr. 2010.
- [6] R. Cattell, "Scalable sql and nosql data stores," *Acm Sigmod Record*, vol. 39, no. 4, pp. 12–27, 2011.
- [7] "DB-Engines Ranking," <http://db-engines.com/en/ranking/>, [Online; accessed 20-April-2019].
- [8] G. Inc., "Map AWS services to Google Cloud Platform products," <http://cloud.google.com/free/docs/map-aws-google-cloud-platform/>.
- [9] S. Rautmare and D. M. Bhalerao, "Mysql and nosql database comparison for iot application," in *2016 IEEE International Conference on Advances in Computer Applications (ICACA)*, Oct 2016, pp. 235–238.
- [10] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with ycsb," in *Proceedings of the 1st ACM symposium on Cloud computing*. ACM, 2010, pp. 143–154.
- [11] J. Kuhlenskamp, M. Klems, and O. Röss, "Benchmarking scalability and elasticity of distributed database systems," *Proceedings of the VLDB Endowment*, vol. 7, no. 12, pp. 1219–1230, 2014.
- [12] J. Gray, *Benchmark Handbook: For Database and Transaction Processing Systems*. Morgan Kaufmann Publishers Inc., 1992.